

Regular Expressions

Finite Automata define a class of languages. FA are Computational device, a FA reads a string letter by letter and decides to accept or reject it.

Another way to define a language is ~~to~~ by an expression. We start ~~by~~ ^{with} some primitive expressions, each ~~is~~ standing for a simple language. Then we combine already constructed expression by a few simple operations on languages like ' $\cup$ ', ' $\cdot$ ' etc, to get a new expression.

(Expressions here are to be considered entities analogous to ~~the~~ arithmetic expressions constructed using arithmetic operators like $+$, $\times$ etc.)

Following is the inductive definitions of Regular Expressions (R.E.)

Def (R.E.)

- (i) ϕ is ~~a~~ a R.E.
- (ii) ϵ is a R.E.
- (iii) For each letter $a \in \Sigma$ ' a ' is R.E.
- (iv) if R, S are R.E. then $R+S$, $R \cdot S$ are also R.E.
- (v) If R is a R.E. then R^* is also R.E.

From our discussion before, each R.E. is expected to denote a language. We define this mapping from R.E. to languages now.

Defn

Let R be a R.E. we define $L(R)$, the language associated with R as follows.

(i) $L(\phi) = \phi$, $L(\epsilon) = \{\epsilon\}$, $L(a) = \{a\}$, $\forall a \in \Sigma$.

(ii) $L(R+S) = L(R) \cup L(S)$ (union)

(iii) $L(R \cdot S) = L(R) \cdot L(S)$ (concatenation)

(iv) $L(R^*) = (L(R))^*$ (Kleene closure)

Operations $\cup$, $\cdot$, $*$ on the right side of equality are semantic operations on languages as written in brackets

Examples

(i) $(a+b)^*$ is the set of all ^(finite) words constructed from letters a, b .

(ii) $(a+b)^* aab (a+b)^*$ is an expression that defines a language over $\{a, b\}^*$ ~~set~~ which has only those words that have a subword "aab".

(iii) $(1 + 01 + 001)^*$ is the set of 0-1 words which do not have three consecutive 0's and end in a '1'.

Exercise: Show that $L((1 + 01 + 001)^*)$ coincides with its English description. In particular, show that any string with the English description is in the language denoted by the regular expression above.

(Hint: Do an induction on length of the string, consider cases based on various suffixes for this string)

(iv) $(1 + 01 + 001)^*(\epsilon + 0 + 00)$ denotes the set of all 0-1 strings ~~with~~ which do not have three consecutive 0's.

Defn

The class of ~~regular~~ languages denoted by regular expression over alphabet Σ are called regular languages over Σ .

Lemma 1

All regular languages are recognizable by a DFA.

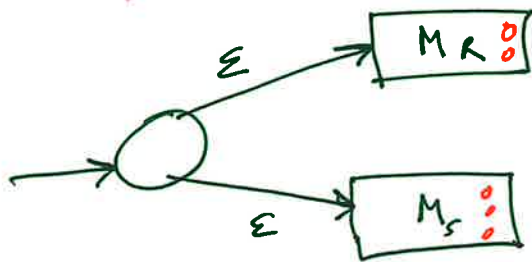
Proof:

By defn $L(R)$, where R is a R.E.

(i) ϕ , $\{\epsilon\}$, $\{a\}$ are clearly recognizable by a DFA.

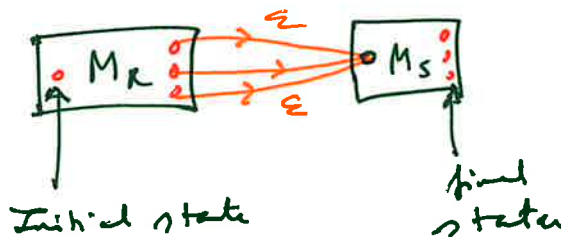
We have already seen that languages recognized by DFA are closed under $\cup$, $\cap$ and $*$, so clauses (i)-(iv) are taken care of, still we show explicitly the finite automata (non-det with ϵ transitions) constructed in each case.

(ii)

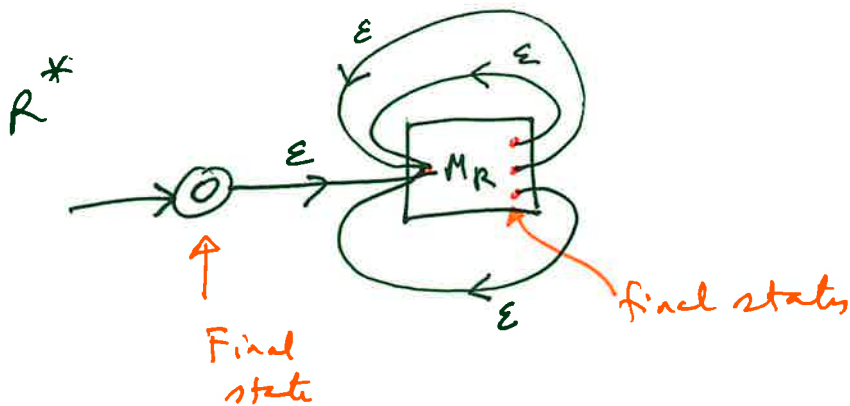


Final states shown by red circles.

$R + S$

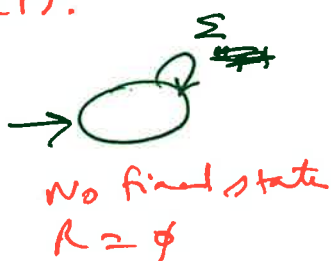


$R . S$

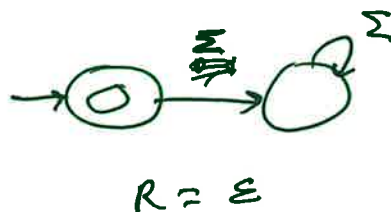


R^*

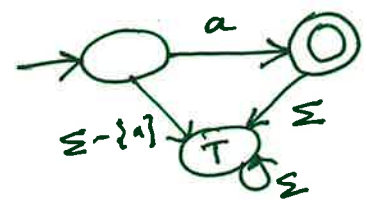
For completeness sake, we also present trivial automata for Case (i).



No final state
 $R = \emptyset$



$R = \epsilon$



$R = a$



Remark: It can be seen that the no. of states in the NFA constructed in Lemma 1 is linear in the size of the regular expression (in fact $\leq 3 \cdot |R|$).

Converse of Lemma 1 is also true.

Lemma 2:

DFA ~~define~~^{recognize} only regular languages.

Proof:

We show how to construct a R.E. ϕ_M from a DFA M (in fact, NFA with ϵ transitions ~~may~~ are allowed as M) s.t. $L(M) = L(\phi_M)$.

Let M be a NFA with ϵ transitions, with states numbered as $\{q_1, q_2, \dots, q_n\}$.

Let R_{ij}^k , where $1 \leq i, j \leq n$ and $0 \leq k \leq n$

stand for the language of words on paths from i to j which ~~to~~ go through ~~intermediate~~ intermediate vertices (other than i, j) $\leq k$ only.

We define regular expression for R_{ij}^k by ~~to~~ induction on k .

Base Case: $k=0$

let r_{ij} be the sum of labels on ~~of~~ edges (q_i, q_j) in the state diagram of M .

$$R_{ij}^0 = \begin{cases} r_{ij} & \text{if } i \neq j \\ r_{ij} + \varepsilon & \text{if } i = j \end{cases}$$

Induction Step:

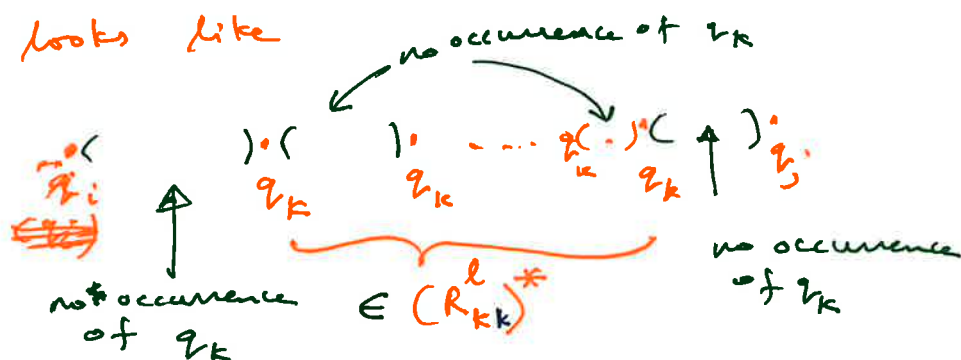
$k = l+1$

$$R_{ij}^k = R_{ij}^{l+1} = R_{ij}^l + R_{i, l+1}^l (R_{l+1, l+1}^l)^* R_{l+1, j}^l$$

For correctness, ~~' $\subseteq$ ' is obvious~~

Containment of RHS in LHS (' $\supseteq$ ') is obvious as every ^{underlying} path on RHS is of the type permitted by LHS. (I.H. is used)

for containment in the other direction ' $\subseteq$ ', let ~~the~~ p be any path of type R_{ij}^k then there are two cases, either p is of type R_{ij}^l (that is, it does not go through $l+1$) or it looks like



Any word corresponding to such a p is in the language of R.E. $R_{i, l+1}^l (R_{l+1, l+1}^l)^* R_{l+1, j}^l$ using induction hypo.

This completes the induction step and the construction of the regular expressions R_{ij}^k .

Finally let q_i be the initial state and let $q_{l_1}, \dots, q_{l_s}$ be final states of M . Then

$$L(M) = L(\hat{R}_{i, l_1}^n + \dots + \hat{R}_{i, l_s}^n)$$

□

$$(\text{we take } \phi_M = \hat{R}_{i, l_1}^n + \dots + \hat{R}_{i, l_s}^n)$$

Remarks:

In the above proof, I have used R_{ij}^k for both a language and a regular expression defining this language. This is not uncommon, though in class I used L_{ij}^k for the language and R_{ij}^k for the R.E. defining this language.

With the above two lemmas, we now know that the class of languages recognized by DFA, NFA, NFA with ϵ -transitions and languages defined by R.E., are all the same. This class is called Regular languages.

Some authors define regular languages to be the ~~class of~~ languages recognized by DFA. In that case, the wordings of Lemma 1 and 2 need to be changed (content and proofs remain the same).